

POLITECNICO
MILANO 1863

SCUOLA DI INGEGNERIA INDUSTRIALE
E DELL'INFORMAZIONE

Data Information Quality Project

Completeness and Distinctness Quality Issues Analyses on a Regression Problem

Author(s): **Irfan Cela - 10694934**

Bianca C. Savoiu Marinas - 10684465

Project ID: **27**

Academic Year: 2023-2024

Contents

Contents **i**

1 Assigned Data quality issues and Machine Learning task **1**
1.1 Problem description 1
1.2 Data quality dimensions 1
1.3 Machine learning task and models 1

2 Setup choices **3**
2.1 Data generation 3
2.2 Missingness mechanisms 3
2.3 Imputers and native handling 4
2.4 Completeness experiments 4
2.5 Distinctness experiments 4

3 Pipeline implementation **6**
3.1 The evaluation protocol 6
3.2 Gaussian process kernel 6
3.3 Differences from the original study 7

4 Results **9**
4.1 Completeness 9
4.1.1 C0: baseline with complete data 9
4.1.2 C1: MCAR with low percentages 9
4.1.3 C2: MCAR with high percentages 10
4.1.4 C3: fixed MCAR on a single feature 15
4.1.5 C4: incremental MCAR per feature with growing sample size 16
4.1.6 C5: MAR 17
4.1.7 C6: MNAR upper tail (self-masking) 18
4.1.8 C7: MCAR in the most informative features 18

4.1.9 C8: MCAR in the less informative features 20
4.1.10 C9: MCAR mixing informative and non-informative features 20
4.1.11 C10: MCAR on all features combined with MNAR on one feature . 21
4.2 Distinctness 22
4.2.1 D1: mid-high distinctness 24
4.2.2 D2: mid-low distinctness 25
4.2.3 D3: fixed distinctness, increasing sample size 26
4.2.4 D4: different percentages among features 26
4.2.5 D8: high and low distinctness combined 28
4.2.6 D9: distinctness on the most informative features 28
4.2.7 D10: distinctness on the less informative features 29
4.2.8 D11: duplicate records 31
4.2.9 D5: random noise on the features (accuracy) 32
4.2.10 D6: binning one feature (precision) 33
4.2.11 D7: outliers (validity) 34

5 Conclusions and limitations **36**
5.1 Conclusions 36
5.2 Limitations 37

Bibliography **38**

1 | Assigned Data quality issues and Machine Learning task

1.1. Problem description

This report studies how two data quality issues, completeness and distinctness, affect a regression task. The data are synthetic: a regression dataset is generated, errors are injected on purpose ("pollution"), the polluted data are handled by simple cleaning steps where this makes sense, and the effect on the prediction error is measured. The report presents the current study. The original implementation is preserved in the repository history (git tag `pre-flatten`), and Section 3.3 explains why the current protocol was needed and which original conclusions survive.

1.2. Data quality dimensions

Following Batini and Scannapieco [5], *completeness* is the extent to which the data contain all the values that should be present, and in a table it is measured through the share of missing values. *Distinctness* (or uniqueness) is the absence of repetition: at record level it concerns duplicate records referring to the same entity, and at column level it is the ratio between the number of distinct values and the number of rows. The original study reduced the column-level ratio; this study additionally studies duplicate records, which is the more standard meaning of the dimension.

1.3. Machine learning task and models

The task is regression on a numeric target. All models are taken from `scikit-learn` [6]. The six study models are:

- `LinearRegressor`: a Ridge regression with the default penalty (the label of the original study is kept);

- `BayesianRidge`;
- `GaussianProcessRegressor` (labelled `GPRegressor` in the tables), with a constant times RBF plus white-noise kernel [7];
- `LinearSVR` (labelled `SVMRegressor`);
- `KNeighborsRegressor` (labelled `KNNRegressor`);
- `MLPRegressor`.

In the completeness experiments only, `HistGradientBoostingRegressor` (labelled `HGBRegressor`) is added as a reference model. It handles missing values natively, so it shows what is achievable without imputation. It is not one of the six study models and it is never counted as an imputation method.

The error measure is the normalised RMSE, nRMSE = RMSE/std(y), where y is the clean target. A value of 1 corresponds to a predictor that always outputs the mean of the target, and the irreducible noise of the data sets a floor of about 0.2 (see Section 2.1).

2 | Setup choices

2.1. Data generation

The data come from `make_regression` of `scikit-learn` with Gaussian features. The target is the signal plus Gaussian noise whose standard deviation is 0.2 times the standard deviation of the signal. The clean nRMSE of a correct model therefore cannot go below about 0.2, which gives a known reference (the original data were noise free, so the clean error was almost zero and a pollution effect could not be compared with an irreducible error). The shapes used in the notebooks are: 1000 rows and 3 informative features for most experiments; 10 features of which 5 informative for C3 and 10 informative features for C10; 5 features with 3 informative for C4; 9 features with 4 informative for C7–C9 and D4, D9, D10. Experiments with a sample-size axis (C0, C4, D3) regenerate the data for every size from 100 to 1000 rows. All features are independent, which matters for the interpretation of the results.

2.2. Missingness mechanisms

Following Rubin [2] and Little and Rubin [3], a value can be missing completely at random (MCAR), at random (MAR) or not at random (MNAR):

- **MCAR**: the probability that a value is missing does not depend on any data. Here each cell is removed independently with the requested probability.
- **MAR**: the probability of missingness depends only on observed values. Here feature 2 is removed where feature 1 is above its $(1-x)$ quantile, so the rate is x and driven by an observed feature.
- **MNAR**: the probability of missingness depends on the missing value itself. Here a feature is removed when its own value exceeds its $(1-x)$ quantile (self-masking of the upper tail).

2.3. Imputers and native handling

Imputation is an explicit experimental condition, always fitted on the training fold only:

- `mean` (the default baseline), `median` and `most_frequent` (degenerate on continuous data);
- `std_constant`: each column is filled with its standard deviation. This is a naive constant that reproduces the method of the original study; it is not a recognised imputation method;
- `knn` (features are scaled first) and `iterative` (chained regressions with a Bayesian ridge estimator);
- `mean_indicator`: mean imputation plus one binary indicator column per incomplete feature, which Josse et al. [4] show to be useful when missingness is informative;
- `native`: no imputation, only for `HGBRegressor`, which receives the raw NaN values;
- `clean`: the unpolluted data, used as the reference in every experiment.

2.4. Completeness experiments

- C0: baseline with complete data and growing sample size.
- C1, C2: MCAR on all 3 features, low (2.5%–25%) and high (27.5%–50%) missing rates.
- C3: 20% MCAR on one feature, as a function of the feature importance rank.
- C4: MCAR with a different rate per column (0%, 10%, ...) and growing sample size.
- C5: MAR; C6: MNAR (upper tail), both with rates 5%–50%.
- C7, C8, C9: MCAR on the most informative, the less informative, and both groups of features.
- C10: MCAR on all features combined with MNAR on one feature.

2.5. Distinctness experiments

The distinctness experiments of the original study are grouped into three families.

- **A. Feature-value distinctness** (D1–D4, D8–D10). A random existing value is pasted into another cell of a chosen column until the ratio of distinct values to rows falls to the target. This reduces distinctness but, because the pasted value replaces the true one, it is also a value-substitution (accuracy) error. This is why the effect grows with the importance of the polluted feature: a corrupted informative column hurts, a corrupted noise column does not (see D9 and D10).
- **B. Duplicate records** (D11, new). Exact copies of random rows are appended so that they make up the requested share of the table. Three conditions are compared: `duplicated` (group-aware splits so a row and its copies are never split between train and test, and exactly one row per held-out source is scored), `deduplicated` (the same splits, with duplicates dropped from the training fold only) and `naive_split`, a labelled leakage demonstration with a plain shuffled split in which copies of training rows reach the test fold [8].
- **C. Accuracy and validity** (formerly distinctness experiments 5–7). D5 adds Gaussian noise to the features, D6 reduces one feature to a few bins and D7 inserts outliers. They do not create non-distinct values and were relabelled; they are kept for comparison with the original results.

The conditions of families A and C are `polluted` (no imputer) and `clean`.

3 Pipeline implementation

3.1. The evaluation protocol

Each fold fits a `TransformedTargetRegressor` that wraps a `Pipeline` (imputer, `StandardScaler`, model) with a target scaler. The imputer and both scalers are fitted on the training rows only, so nothing is fitted on test rows; this avoids leakage [8]. The scale of nRMSE is fixed by the clean target. A finite-value check raises an error if a NaN would reach a model, so nothing is silently replaced.

Splits are `ShuffleSplit` with 5 folds and a test share of 0.3. Every experiment uses the repetition seeds 0, 1 and 2. Each seed is expanded with `numpy`'s `SeedSequence` into independent seeds for the data generation, the pollution and the splits. The design is paired: for a given seed all conditions and models see the same data, the same pollution mask and the same folds.

The main metric is the test nRMSE. The train nRMSE is also stored, so that the train–test gap is available in the generated gap figures. Folds are first averaged within a seed. The plotted bands and the tables report the mean ± SD over the 3 seeds. These are descriptive spreads across repetitions and not confidence intervals.

3.2. Gaussian process kernel

The Gaussian process uses a constant times RBF plus white-noise kernel [7], with normalised target and a lower bound of 0.1 on the length scale (in standardised feature units) and one optimiser restart. With the default bounds the optimiser could converge to a length scale at the lower limit with a worse marginal likelihood, and the process collapsed to the mean predictor; in a probe on a polluted data set the bounded kernel avoided this collapse, and extra restarts did not fix the problem. Hits of the bounds are recorded in the status of each fold.

3.3. Differences from the original study

The original implementation is preserved in the repository history (git tag `pre-flatten`). An audit of it found the following defects.

- **No imputation equalled mean imputation.** The scaler was fitted on the data with NaN ignored and the NaN were then replaced by zero, which in standardised space is the column mean. The "raw" baseline of the original study was therefore mean imputation, and the finding that imputation does not help was partly built in.
- **Leakage.** Imputers and scalers were fitted before the train/test split, so test rows influenced the fill values.
- **MAR and MNAR created no missing values.** The thresholds (for example > 100 on standard Gaussian features) never fired, so those two experiments were complete-data runs.
- **Discarded imputations.** The results of the standard-deviation, KNN and linear-regression imputation were discarded or applied to one column only, so they equalled the raw data.
- **Constants instead of NaN.** The "less informative" and "most informative" experiments wrote a constant value, not a missing value, and the incremental experiment made four of five features entirely missing.
- **Sample-size axes too narrow.** The axes 1000–1090 rows cannot show a size effect. This study uses 100–1000.
- **Distinctness target not reached,** one fixed column polluted, and noise variance used as standard deviation.
- **Gaussian process collapse** which depended on scikit-learn's default kernel (Section 3.2): in the original environment (scikit-learn $\leq$ 1.8) the default `GaussianProcessRegressor()` used fixed hyperparameters, so no optimisation ran and the model was a near noise-free RBF interpolator ($\alpha = 10^{-10}$) with an ill-conditioned kernel matrix, and on polluted data the predictions exploded (errors in the thousands); from scikit-learn 1.9 the default kernel is optimised and, with the default length-scale bounds $(10^{-5}, 10^{5})$, converges to the length scale 10^{-5}, i.e. the mean predictor. A `WhiteKernel` added in an intermediate faithful-fix stage still collapsed on heavily tied data and restarts did not help; this study sets the lower length-scale bound to 0.1 in standardised units. The data were noise free, a single seed was used,

and no clean reference was reported.

The corrections are listed in `library/HISTORY.md`. For the original conclusions the audit gives the following verdicts, checked against the results of this study.

- *Imputation is insufficient, error increases persistently:* not supported by the original runs, but in this study the increase with the missing rate is confirmed (C1, C2), with the explanation that independent features cannot inform the imputer.
- *Missing values in the most informative features are critical:* plausible, but the original run did not create NaN; this study confirms it (C3, C7–C9 against C8).
- *MAR and MNAR do not influence the model:* invalidated; they produced no NaN. Here MAR is mild (C5) and MNAR is harmful with biased imputers (C6).
- *The Gaussian process is more sensitive to missing data:* an artefact of the kernel setup; in this study the process behaves like the linear model on the clean data (C0).
- *Pollution of unimportant features is crucial (distinctness):* not reproduced (D10).
- *Distinctness behaves non-monotonically or is invariant to the percentage:* a consequence of the algorithm missing its target; here the effect saturates (D1, D2).
- *Distinctness of a single column significantly affects the model:* supported only against a clean reference, and the sample-size version (D3) shows no trend.

4 | Results

All numbers in this chapter come from the tables generated for this report. Every table shows the first, a middle and the last point of the x axis (the figures show the full axis) as test nRMSE, mean ± SD over 3 seeds; lower is better and the clean noise floor is about 0.2. In the figures, solid lines are means and bands are the SD over seeds.

4.1. Completeness

4.1.1. C0: baseline with complete data

Data are regenerated for every n (3 informative features). The linear model reaches 0.188 ± 0.015 at $n = 100$ and 0.199 ± 0.004 at $n = 1000$, i.e. the noise floor of about 0.2, and the Gaussian process is at the same level (0.194 and 0.199). The MLP is slightly above (0.220, 0.208) and `KNeighborsRegressor` improves with more data (0.384 at $n = 100$, 0.267 at $n = 1000$), so it is the model that benefits most from sample size and stays above the floor. This is the learning curve that the sample-size experiments C4 and D3 are compared against.

Condition	Model	100	600	1000
clean	LinearRegressor	0.188 ± 0.015	0.197 ± 0.004	0.199 ± 0.004
clean	GPRegressor	0.194 ± 0.015	0.196 ± 0.004	0.199 ± 0.004
clean	KNNRegressor	0.384 ± 0.020	0.292 ± 0.025	0.267 ± 0.005
clean	MLPRegressor	0.220 ± 0.018	0.210 ± 0.004	0.208 ± 0.005

Table 4.1: C0: test nRMSE (lower is better) against sample size n (100, 600, 1000 shown); mean ± SD over 3 seeds.

4.1.2. C1: MCAR with low percentages

Cells of all 3 informative features are removed completely at random (1000 rows). With 2.5% missing cells the mean-imputed linear model already moves from 0.199 (clean) to 0.256 ± 0.008, and reaches 0.543 ± 0.017 at 25%. `median`, `iterative` and `mean_indicator`

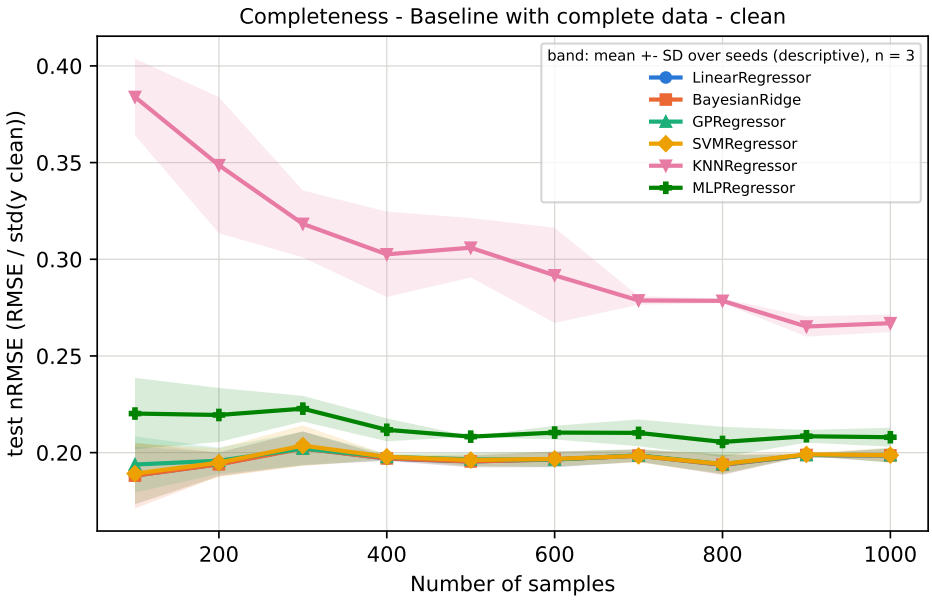


Figure 4.1: C0: test nRMSE of each model, condition `clean`, mean ± SD over 3 seeds.

give practically the same numbers as `mean` (0.256, 0.256 and 0.254 at 2.5%; 0.543, 0.544 and 0.541 at 25%): the three features are generated independently, so the other columns carry no information on the missing one and every reasonable imputer ends at the column centre. `knn` is slightly worse (0.264 and 0.580). The two naive options are clearly worse: `most_frequent`, which is degenerate on continuous data, gives 0.502 and 0.880, and `std_constant` (the method of the original study) gives 0.302 ± 0.027 and 0.659 ± 0.034. `KNeighborsRegressor` shows the same ordering at a higher level ($0.319 \rightarrow 0.616$ with `mean`). The native-NaN `HistGradientBoostingRegressor` behaves like its mean-imputed counterpart (0.296 against 0.305 at 2.5%, 0.601 against 0.599 at 25%).

4.1.3. C2: MCAR with high percentages

The same setting at 27.5%–50% missing cells (`most_frequent` and `std_constant` omitted). The mean-imputed linear model goes from 0.564 ± 0.011 to 0.719 ± 0.006 and `KNeighborsRegressor` from 0.634 to 0.813. `knn` imputation is again a little worse ($0.602 \rightarrow 0.755$ for the linear model), `iterative` is equal to `mean` up to 40% and slightly worse at 50% (0.727 against 0.719), and the missing-value indicator brings no gain (0.718). The native `HistGradientBoostingRegressor` gives $0.627 \rightarrow 0.780$, close to its mean-imputed version ($0.623 \rightarrow 0.774$).

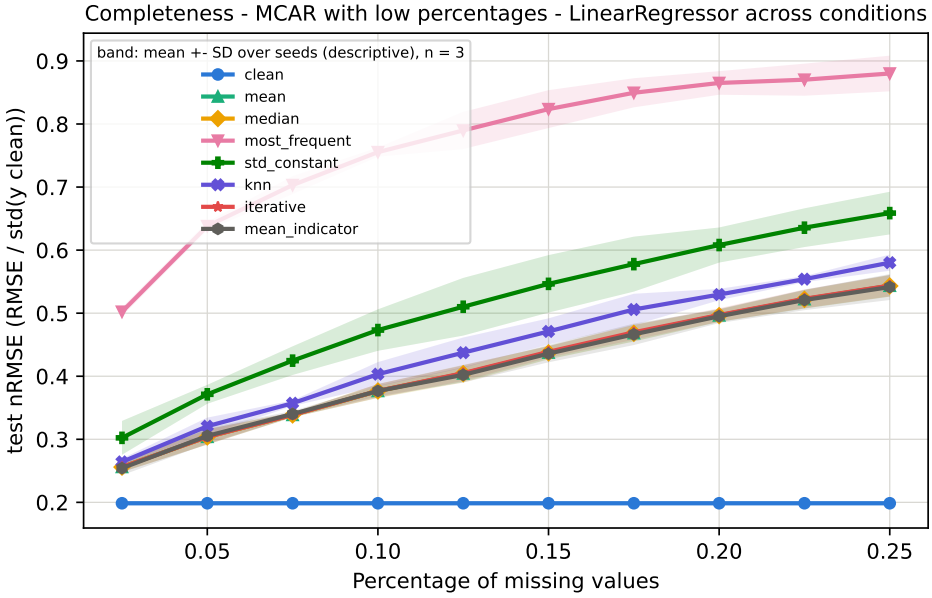


Figure 4.2: C1: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

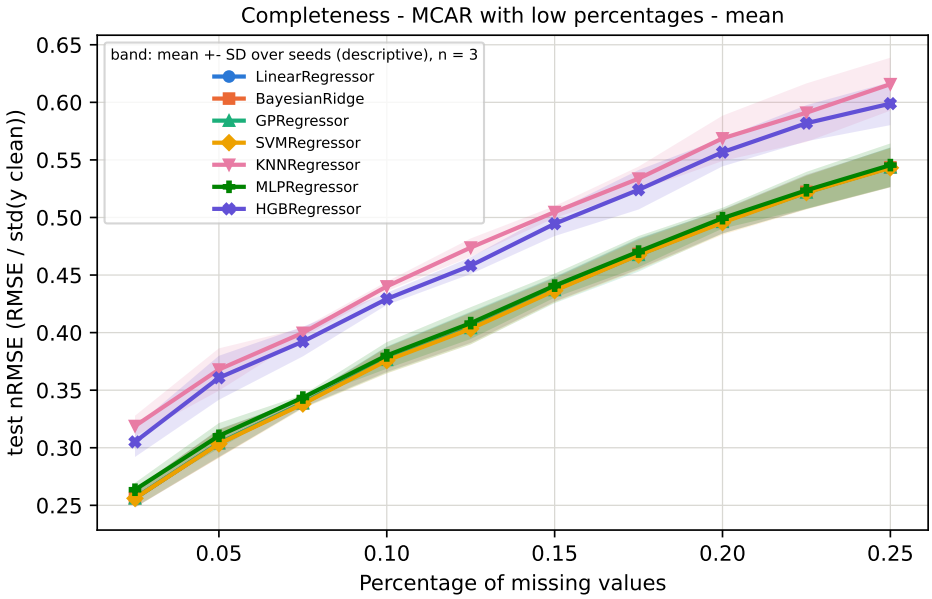


Figure 4.3: C1: test nRMSE of each model, condition `mean`, mean ± SD over 3 seeds.

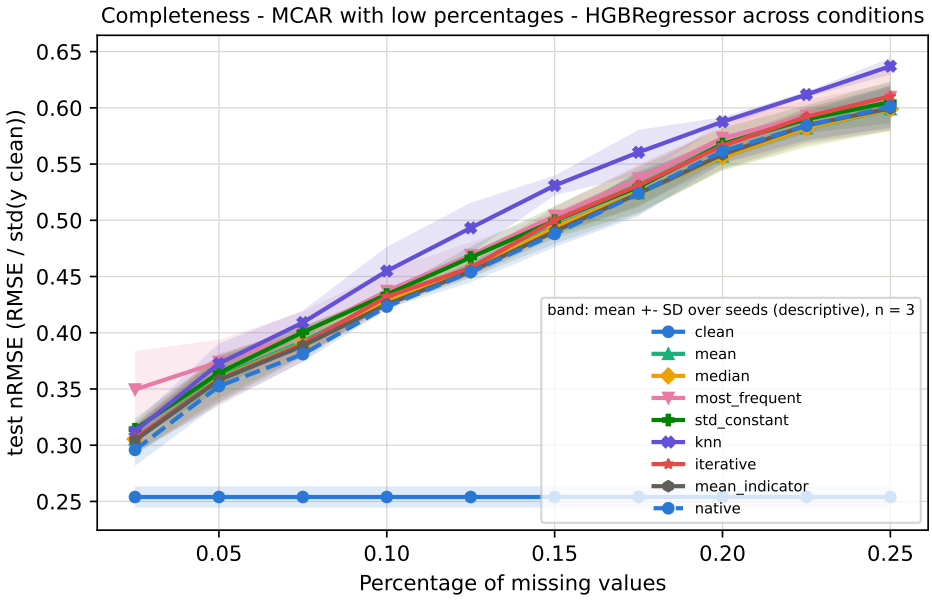


Figure 4.4: C1: test nRMSE of `HGBRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	0.025	0.15	0.25
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004
clean	KNNRegressor	0.267 ± 0.005	0.267 ± 0.005	0.267 ± 0.005
mean	LinearRegressor	0.256 ± 0.008	0.437 ± 0.010	0.543 ± 0.017
mean	KNNRegressor	0.319 ± 0.009	0.505 ± 0.005	0.616 ± 0.023
median	LinearRegressor	0.256 ± 0.008	0.437 ± 0.010	0.543 ± 0.017
median	KNNRegressor	0.319 ± 0.008	0.504 ± 0.008	0.614 ± 0.023
most_frequent	LinearRegressor	0.502 ± 0.007	0.824 ± 0.030	0.880 ± 0.028
most_frequent	KNNRegressor	0.375 ± 0.019	0.561 ± 0.030	0.653 ± 0.038
std_constant	LinearRegressor	0.302 ± 0.027	0.546 ± 0.046	0.659 ± 0.034
std_constant	KNNRegressor	0.359 ± 0.019	0.588 ± 0.029	0.683 ± 0.026
knn	LinearRegressor	0.264 ± 0.005	0.471 ± 0.021	0.580 ± 0.013
knn	KNNRegressor	0.324 ± 0.005	0.531 ± 0.013	0.647 ± 0.012
iterative	LinearRegressor	0.256 ± 0.008	0.439 ± 0.009	0.544 ± 0.017
iterative	KNNRegressor	0.319 ± 0.009	0.507 ± 0.005	0.616 ± 0.021
mean_indicator	LinearRegressor	0.254 ± 0.011	0.436 ± 0.014	0.541 ± 0.021
mean_indicator	KNNRegressor	0.330 ± 0.007	0.511 ± 0.016	0.619 ± 0.029
native	HGBRegressor	0.296 ± 0.014	0.488 ± 0.012	0.601 ± 0.019
mean	HGBRegressor	0.305 ± 0.013	0.495 ± 0.011	0.599 ± 0.019

Table 4.2: C1: test nRMSE (lower is better) against missing rate (0.025, 0.15, 0.25 shown); mean ± SD over 3 seeds.

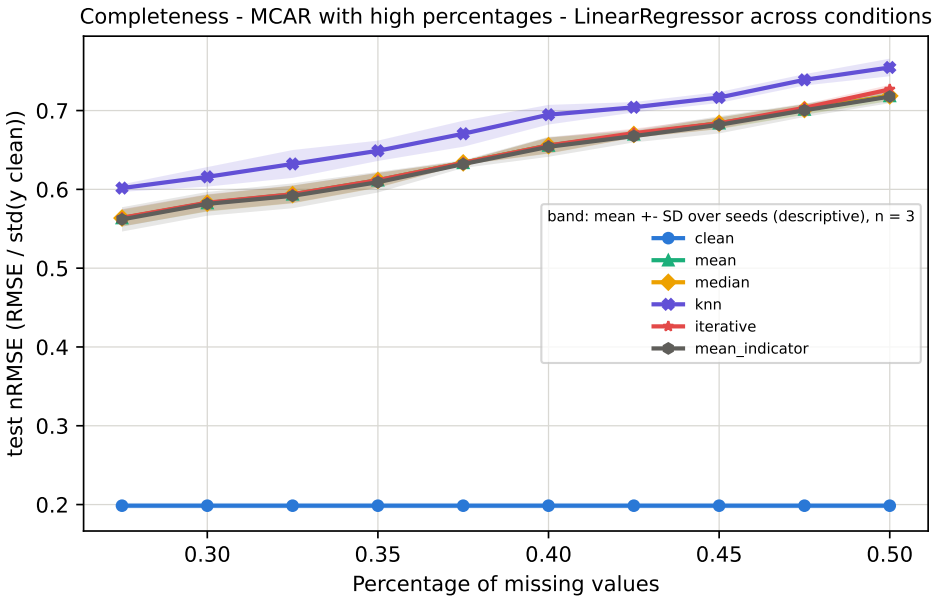


Figure 4.5: C2: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

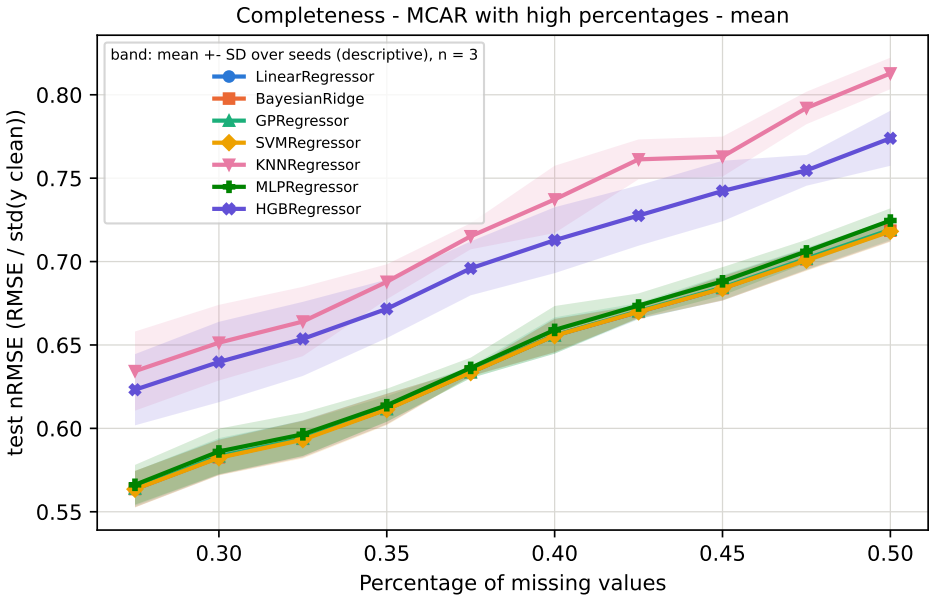


Figure 4.6: C2: test nRMSE of each model, condition `mean`, mean ± SD over 3 seeds.

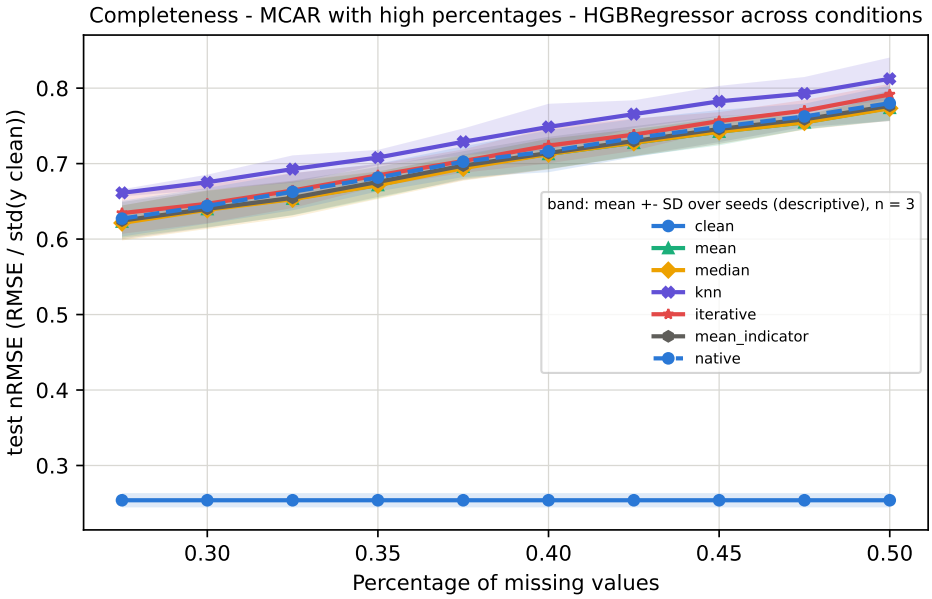


Figure 4.7: C2: test nRMSE of `HGBRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	0.275	0.4	0.5
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004
clean	KNNRegressor	0.267 ± 0.005	0.267 ± 0.005	0.267 ± 0.005
mean	LinearRegressor	0.564 ± 0.011	0.656 ± 0.010	0.719 ± 0.006
mean	KNNRegressor	0.634 ± 0.024	0.737 ± 0.020	0.813 ± 0.009
median	LinearRegressor	0.564 ± 0.010	0.655 ± 0.010	0.719 ± 0.006
median	KNNRegressor	0.636 ± 0.025	0.743 ± 0.014	0.803 ± 0.004
knn	LinearRegressor	0.602 ± 0.005	0.695 ± 0.012	0.755 ± 0.011
knn	KNNRegressor	0.667 ± 0.018	0.757 ± 0.024	0.820 ± 0.012
iterative	LinearRegressor	0.564 ± 0.011	0.656 ± 0.010	0.727 ± 0.006
iterative	KNNRegressor	0.633 ± 0.026	0.731 ± 0.020	0.797 ± 0.008
mean_indicator	LinearRegressor	0.562 ± 0.015	0.654 ± 0.013	0.718 ± 0.008
mean_indicator	KNNRegressor	0.635 ± 0.041	0.735 ± 0.020	0.808 ± 0.010
native	HGBRegressor	0.627 ± 0.024	0.716 ± 0.028	0.780 ± 0.023
mean	HGBRegressor	0.623 ± 0.021	0.713 ± 0.020	0.774 ± 0.017

Table 4.3: C2: test nRMSE (lower is better) against missing rate (0.275, 0.4, 0.5 shown); mean ± SD over 3 seeds.

4.1.4. C3: fixed MCAR on a single feature

20% of one column is removed; there are 10 features, ranks 0–4 are informative and ranks 5–9 are noise. Missingness in the most important feature (rank 0) raises the mean-imputed linear model from 0.202 (clean) to 0.365 ± 0.033, whereas for the noise features at rank 5 and 9 it stays at 0.202, identical to the clean value. `median` and `iterative` imputation reproduce these values (0.365, 0.366). The native-NaN boosting model shows the same pattern (0.445 at rank 0 against 0.301 and 0.302 at ranks 5 and 9). Only informative features matter.

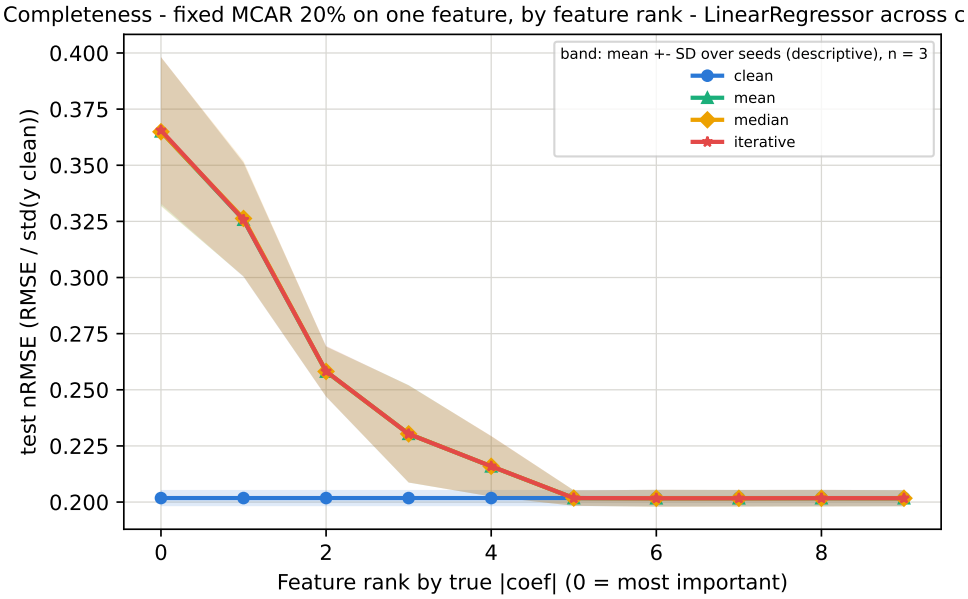


Figure 4.8: C3: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	0	5	9
clean	LinearRegressor	0.202 ± 0.004	0.202 ± 0.004	0.202 ± 0.004
mean	LinearRegressor	0.365 ± 0.033	0.202 ± 0.003	0.202 ± 0.004
median	LinearRegressor	0.365 ± 0.033	0.202 ± 0.003	0.202 ± 0.004
iterative	LinearRegressor	0.366 ± 0.033	0.202 ± 0.003	0.202 ± 0.004
mean	HGBRegressor	0.446 ± 0.025	0.299 ± 0.013	0.299 ± 0.014
native	HGBRegressor	0.445 ± 0.019	0.301 ± 0.015	0.302 ± 0.013

Table 4.4: C3: test nRMSE (lower is better) against feature rank by true |coefficient| (0, 5, 9 shown); mean ± SD over 3 seeds.

4.1.5. C4: incremental MCAR per feature with growing sample size

Column i loses $10i\%$ of its cells (5 features, 3 informative) while n grows. The linear model with mean imputation gives 0.428 ± 0.179, 0.439 ± 0.152 and 0.525 ± 0.097 at $n = 100$, 600, 1000, against 0.188, 0.198 and 0.200 for the clean data. There is no reliable trend with n given the SD over seeds, which is large (up to 0.179). The boosting model shows the same absence of a trend (0.668, 0.512, 0.586 with `mean`). The polluter uses the same seed for every column, so the missing-row sets are nested; this is kept and listed as a limitation.

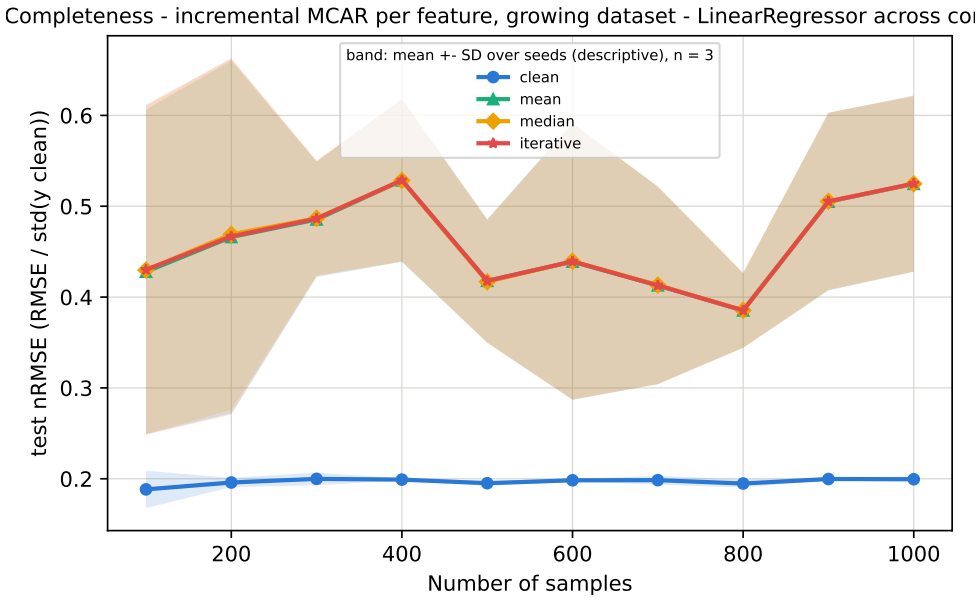


Figure 4.9: C4: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	100	600	1000
clean	LinearRegressor	0.188 ± 0.021	0.198 ± 0.002	0.200 ± 0.003
mean	LinearRegressor	0.428 ± 0.179	0.439 ± 0.152	0.525 ± 0.097
median	LinearRegressor	0.430 ± 0.181	0.439 ± 0.152	0.525 ± 0.096
iterative	LinearRegressor	0.430 ± 0.182	0.439 ± 0.152	0.525 ± 0.097
mean	HGBRegressor	0.668 ± 0.170	0.512 ± 0.166	0.586 ± 0.109
native	HGBRegressor	0.604 ± 0.140	0.506 ± 0.160	0.585 ± 0.106

Table 4.5: C4: test nRMSE (lower is better) against sample size n (100, 600, 1000 shown); mean ± SD over 3 seeds.

4.1.6. C5: MAR

Feature 2 is removed where feature 1 exceeds its $(1 - x)$ quantile, so the realised rate is x and depends only on an observed value. The effect is mild at the tested rates, but only one column is affected, so this is not a like-for-like comparison with MCAR on all features (C1, C2); the closest one-column reference is C3. The mean-imputed linear model gives 0.217 ± 0.023, 0.269 ± 0.085 and 0.309 ± 0.128 at 5%, 30% and 50% (clean 0.199). `median`, `iterative` and `mean_indicator` are indistinguishable from `mean` (for example 0.309, 0.309 and 0.310 at 50%). The SD over seeds grows with the rate, so the seeds disagree on how harmful the mechanism is. Native boosting gives 0.263, 0.320, 0.362.

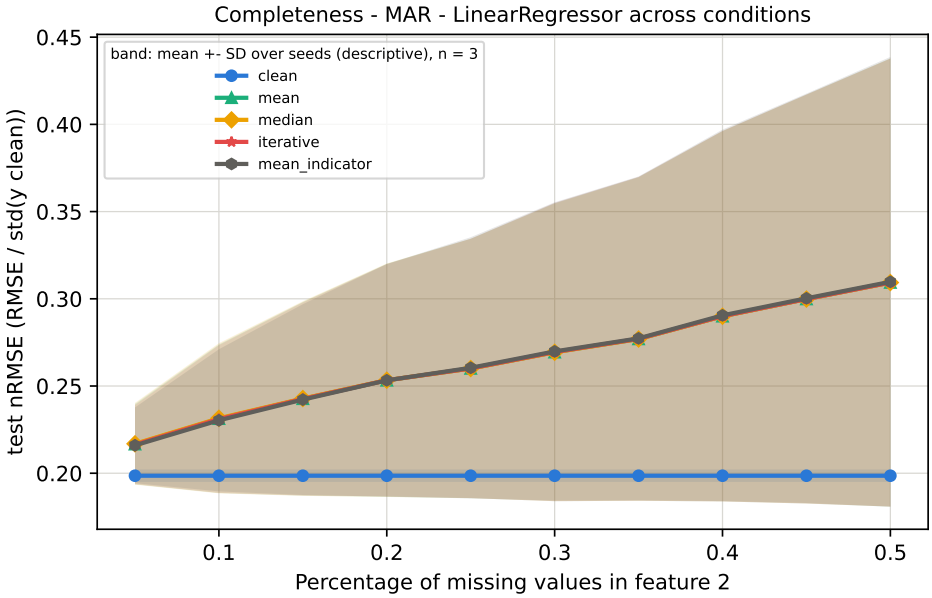


Figure 4.10: C5: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	0.05	0.3	0.5
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004
mean	LinearRegressor	0.217 ± 0.023	0.269 ± 0.085	0.309 ± 0.128
median	LinearRegressor	0.217 ± 0.023	0.269 ± 0.085	0.309 ± 0.128
iterative	LinearRegressor	0.216 ± 0.023	0.269 ± 0.086	0.309 ± 0.128
mean_indicator	LinearRegressor	0.216 ± 0.022	0.270 ± 0.086	0.310 ± 0.129
mean	HGBRegressor	0.265 ± 0.024	0.320 ± 0.093	0.363 ± 0.139
native	HGBRegressor	0.263 ± 0.023	0.320 ± 0.090	0.362 ± 0.138

Table 4.6: C5: test nRMSE (lower is better) against missing rate (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

4.1.7. C6: MNAR upper tail (self-masking)

Feature 1 is removed when its own value exceeds its $(1 - x)$ quantile, so the largest values vanish. Here the imputer matters. Mean imputation of the linear model gives 0.422±0.048, 0.664±0.092 and 0.740±0.112, and `iterative` imputation is identical (0.422, 0.664, 0.740): both place the missing values at the centre of the observed (truncated) distribution and are biased. Adding the missing-value indicator recovers a large part of the loss (0.206 ± 0.003, 0.301 ± 0.024, 0.388 ± 0.041), because the indicator tells the model that the value was in the upper tail. The native boosting model behaves like the indicator variant (0.257, 0.350, 0.443) and is better than its mean-imputed version at 5% (0.292) but similar at 30% and 50% (0.358, 0.452), within one SD.

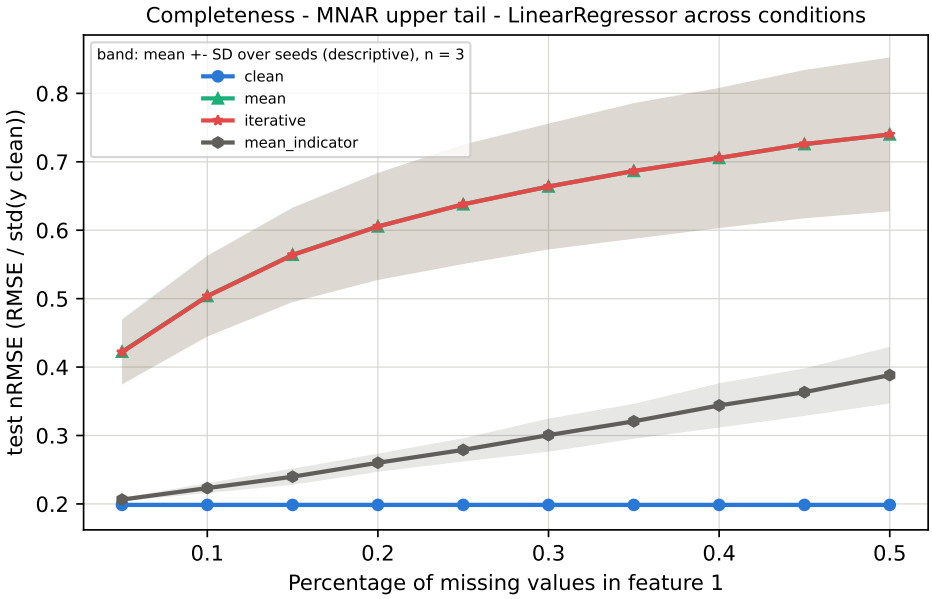


Figure 4.11: C6: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

4.1.8. C7: MCAR in the most informative features

The 3 most important of 9 features (4 informative) lose 5%–50% of their cells. The mean-imputed linear model gives 0.300 ± 0.021, 0.578 ± 0.038 and 0.705 ± 0.035 (clean 0.200); `median` and `iterative` are equal. Native boosting is slightly worse (0.373, 0.647, 0.792).

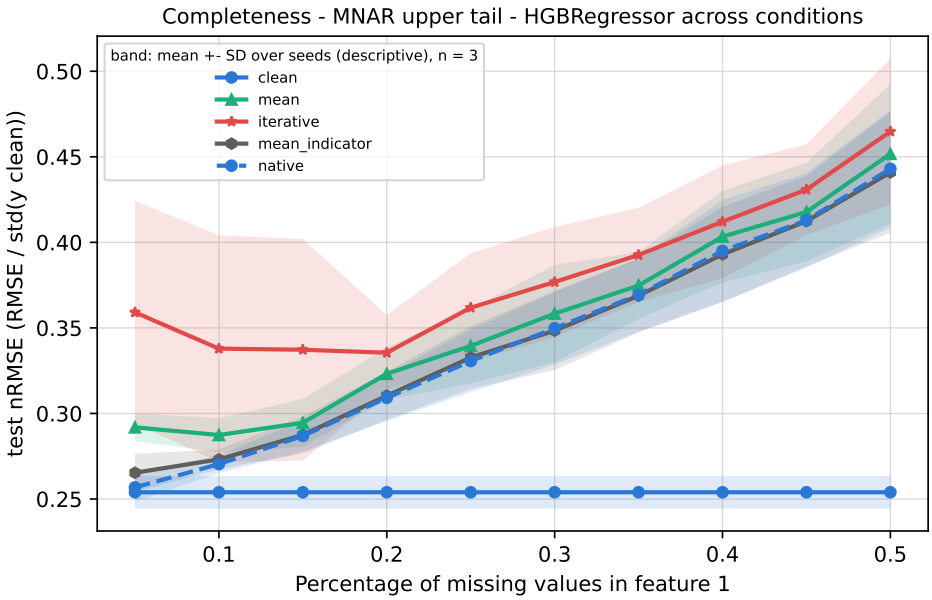


Figure 4.12: C6: test nRMSE of `HGBRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	0.05	0.3	0.5
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004
mean	LinearRegressor	0.422 ± 0.048	0.664 ± 0.092	0.740 ± 0.112
iterative	LinearRegressor	0.422 ± 0.047	0.664 ± 0.092	0.740 ± 0.112
mean_indicator	LinearRegressor	0.206 ± 0.003	0.301 ± 0.024	0.388 ± 0.041
mean	HGBRegressor	0.292 ± 0.008	0.358 ± 0.028	0.452 ± 0.041
mean_indicator	HGBRegressor	0.265 ± 0.011	0.348 ± 0.023	0.441 ± 0.035
native	HGBRegressor	0.257 ± 0.008	0.350 ± 0.021	0.443 ± 0.034

Table 4.7: C6: test nRMSE (lower is better) against missing rate (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

Condition	Model	0.05	0.3	0.5
clean	LinearRegressor	0.200 ± 0.005	0.200 ± 0.005	0.200 ± 0.005
mean	LinearRegressor	0.300 ± 0.021	0.578 ± 0.038	0.705 ± 0.035
median	LinearRegressor	0.301 ± 0.020	0.578 ± 0.037	0.705 ± 0.034
iterative	LinearRegressor	0.301 ± 0.021	0.578 ± 0.039	0.709 ± 0.037
mean	HGBRegressor	0.372 ± 0.014	0.651 ± 0.053	0.796 ± 0.027
native	HGBRegressor	0.373 ± 0.019	0.647 ± 0.051	0.792 ± 0.029

Table 4.8: C7: test nRMSE (lower is better) against missing rate (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

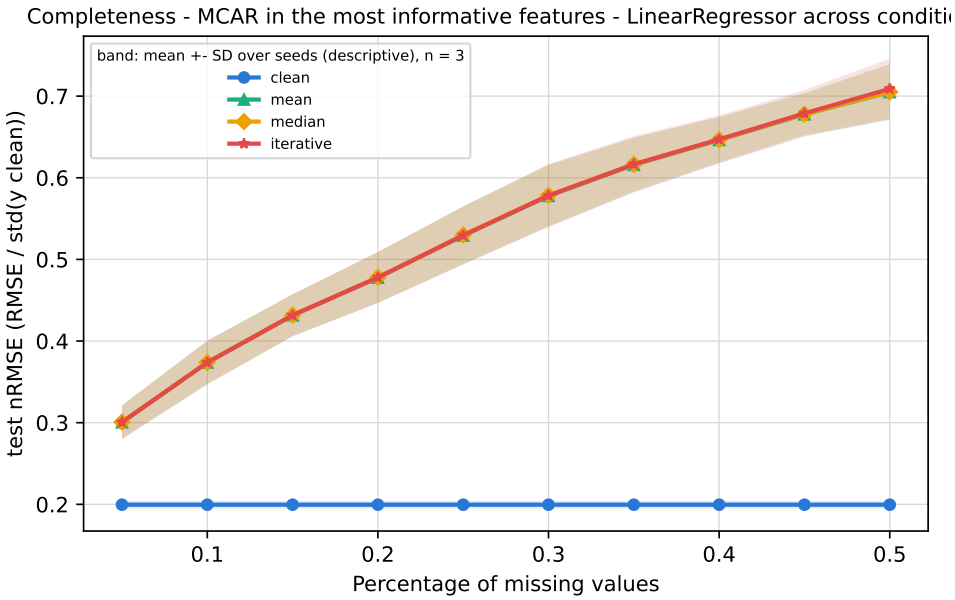


Figure 4.13: C7: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

4.1.9. C8: MCAR in the less informative features

The same experiment for the 3 least important features. The linear model with any imputer stays at 0.199 for all rates, the clean value, and boosting stays at about 0.27 (0.270, 0.269, 0.268 with `mean`). Missing values in uninformative columns are harmless.

Condition	Model	0.05	0.3	0.5
clean	LinearRegressor	0.200 ± 0.005	0.200 ± 0.005	0.200 ± 0.005
mean	LinearRegressor	0.199 ± 0.005	0.199 ± 0.005	0.199 ± 0.004
median	LinearRegressor	0.199 ± 0.005	0.199 ± 0.005	0.199 ± 0.004
iterative	LinearRegressor	0.199 ± 0.005	0.199 ± 0.005	0.199 ± 0.004
mean	HGBRegressor	0.270 ± 0.008	0.269 ± 0.007	0.268 ± 0.008
native	HGBRegressor	0.271 ± 0.010	0.272 ± 0.008	0.269 ± 0.007

Table 4.9: C8: test nRMSE (lower is better) against missing rate (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

4.1.10. C9: MCAR mixing informative and non-informative features

Both sets (the 3 most and the 3 least important features, ranked once on the clean data) lose cells. The numbers coincide with C7 (0.301, 0.577, 0.705 for the mean-imputed linear

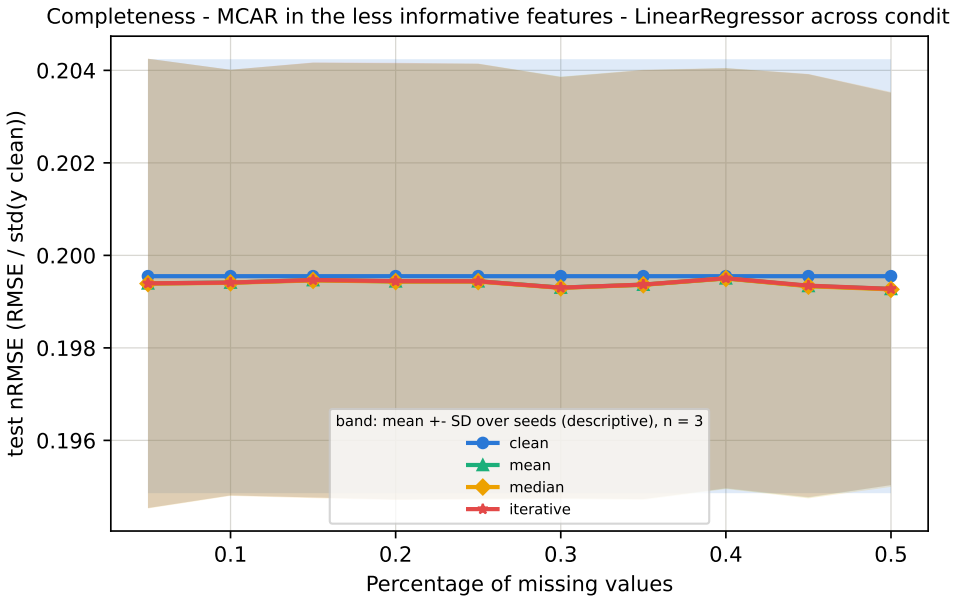


Figure 4.14: C8: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

model), showing that the additional uninformative columns add nothing. Boosting with `mean` gives 0.372, 0.654, 0.801. Because the groups of C7 and C9 share row sets, the two experiments are not independent evidence.

Condition	Model	0.05	0.3	0.5
clean	LinearRegressor	0.200 ± 0.005	0.200 ± 0.005	0.200 ± 0.005
mean	LinearRegressor	0.301 ± 0.020	0.577 ± 0.037	0.705 ± 0.033
median	LinearRegressor	0.301 ± 0.020	0.578 ± 0.036	0.705 ± 0.032
iterative	LinearRegressor	0.301 ± 0.020	0.577 ± 0.037	0.709 ± 0.035
mean	HGBRegressor	0.372 ± 0.016	0.654 ± 0.051	0.801 ± 0.032
native	HGBRegressor	0.377 ± 0.022	0.659 ± 0.046	0.807 ± 0.035

Table 4.10: C9: test nRMSE (lower is better) against missing rate (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

4.1.11. C10: MCAR on all features combined with MNAR on one feature

All 10 features lose a share x of their cells at random and feature 0 additionally loses its upper-tail 25%. The mean-imputed linear model gives 0.346 ± 0.042, 0.595 ± 0.032, 0.748 ± 0.012. The indicator helps at 5% (0.310 ± 0.014) but not at higher rates (0.589

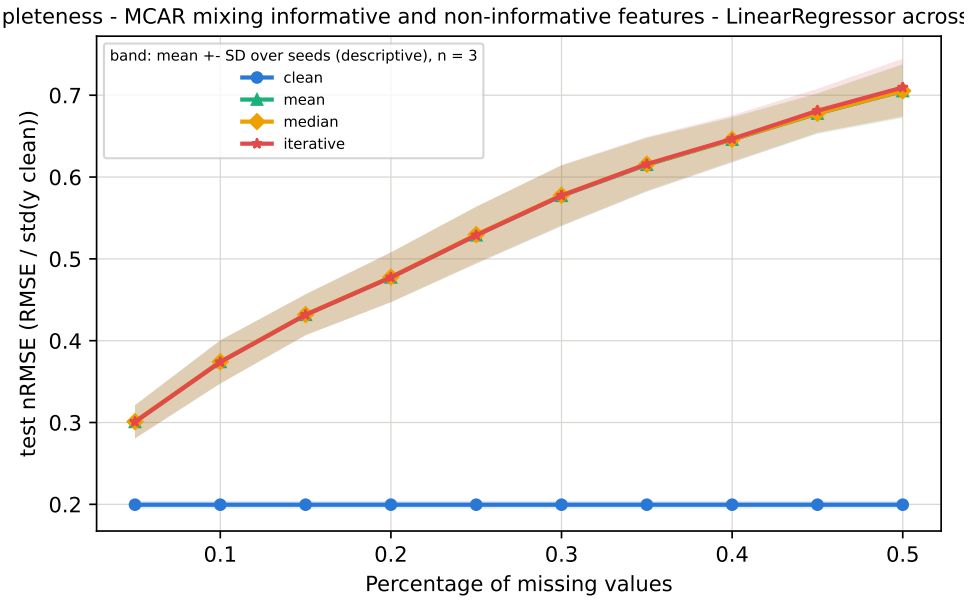


Figure 4.15: C9: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

at 30%, 0.749 at 50%), presumably because the MCAR component dominates. Note that feature 0 has a realised missing rate of about $0.25 + 0.75x$ (0.29 at 5%, 0.63 at 50%), so the rate axis is not the missing rate of feature 0. `iterative` imputation is clearly worse at 50% (0.877 ± 0.020). Boosting is similar to mean imputation with native NaN handling (0.435, 0.687, 0.841).

Condition	Model	0.05	0.3	0.5
clean	LinearRegressor	0.200 ± 0.003	0.200 ± 0.003	0.200 ± 0.003
mean	LinearRegressor	0.346 ± 0.042	0.595 ± 0.032	0.748 ± 0.012
iterative	LinearRegressor	0.346 ± 0.042	0.596 ± 0.033	0.877 ± 0.020
mean_indicator	LinearRegressor	0.310 ± 0.014	0.589 ± 0.027	0.749 ± 0.009
mean	HGBRegressor	0.433 ± 0.033	0.682 ± 0.035	0.833 ± 0.006
mean_indicator	HGBRegressor	0.429 ± 0.029	0.677 ± 0.035	0.838 ± 0.015
native	HGBRegressor	0.435 ± 0.027	0.687 ± 0.033	0.841 ± 0.014

Table 4.11: C10: test nRMSE (lower is better) against MCAR rate (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

4.2. Distinctness

Family A: feature-value distinctness

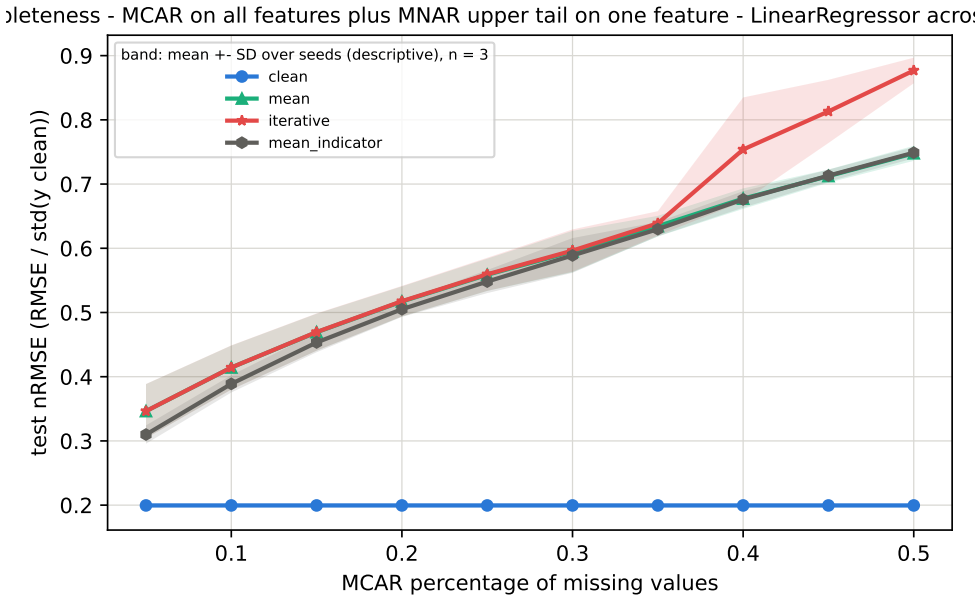


Figure 4.16: C10: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

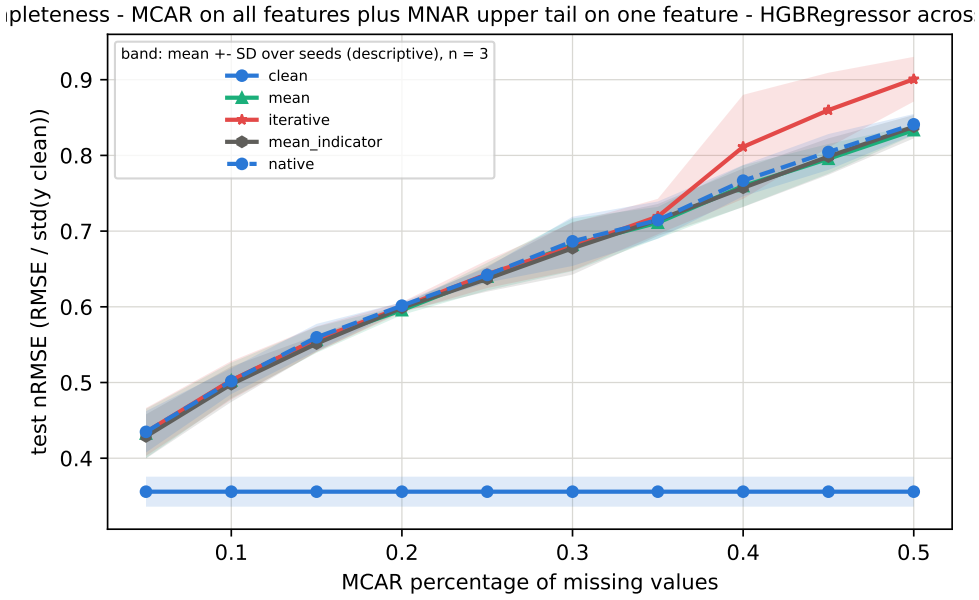


Figure 4.17: C10: test nRMSE of `HGBRegressor` for each condition, mean ± SD over 3 seeds.

4.2.1. D1: mid-high distinctness

One random feature of 3 loses distinct values. The polluted linear model rises from 0.199 to 0.240 ± 0.020 at 25% and 0.271 ± 0.038 at 50%, and the other models follow the same curve (BayesianRidge 0.271, Gaussian process 0.271, `KNeighborsRegressor` 0.267 → 0.333). The pollution pastes another row's value into a cell, so it is also a value-substitution (accuracy) error.

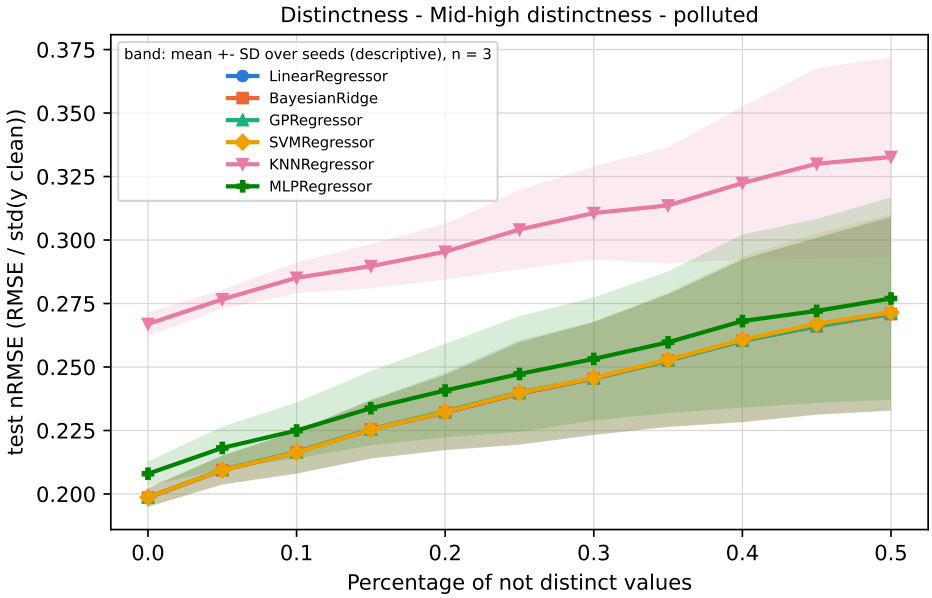


Figure 4.18: D1: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

Condition	Model	0	0.25	0.5
polluted	LinearRegressor	0.199 ± 0.004	0.240 ± 0.020	0.271 ± 0.038
polluted	BayesianRidge	0.199 ± 0.004	0.240 ± 0.020	0.271 ± 0.038
polluted	GPRegressor	0.199 ± 0.004	0.240 ± 0.021	0.271 ± 0.038
polluted	SVMRegressor	0.199 ± 0.004	0.240 ± 0.020	0.272 ± 0.039
polluted	KNNRegressor	0.267 ± 0.005	0.304 ± 0.016	0.333 ± 0.039
polluted	MLPRegressor	0.208 ± 0.005	0.247 ± 0.023	0.277 ± 0.040
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004

Table 4.12: D1: test nRMSE (lower is better) against fraction of non-distinct values (0, 0.25, 0.5 shown); mean ± SD over 3 seeds.

4.2.2. D2: mid-low distinctness

From 50% to 100% the linear model goes 0.271 ± 0.038, 0.282 ± 0.045, 0.282 ± 0.045: the curve saturates, and at 100% the column is constant, so the models effectively lose one feature. The difference between seeds (± 0.045) presumably reflects which random feature was polluted.

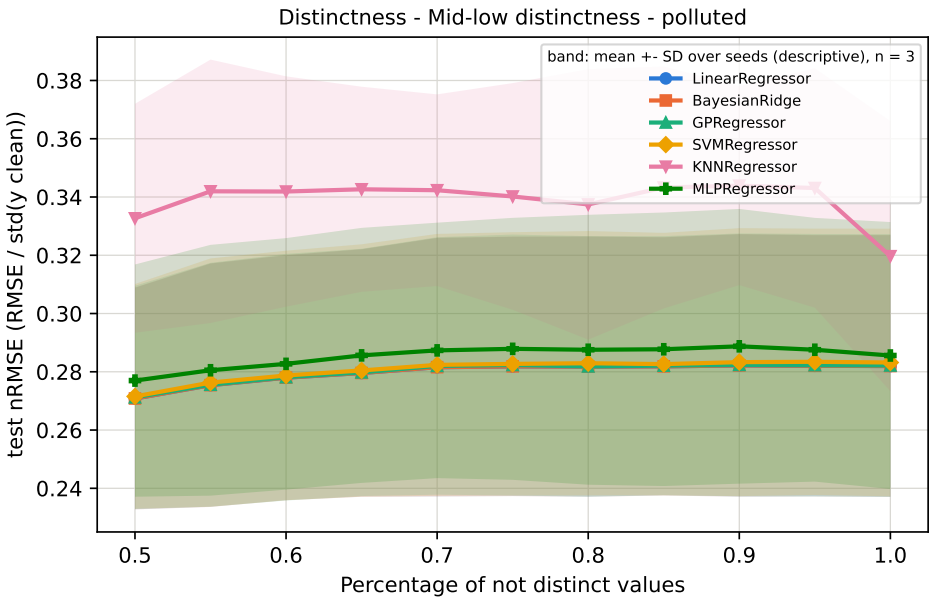


Figure 4.19: D2: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

Condition	Model	0.5	0.75	1
polluted	LinearRegressor	0.271 ± 0.038	0.282 ± 0.045	0.282 ± 0.045
polluted	BayesianRidge	0.271 ± 0.038	0.282 ± 0.045	0.282 ± 0.045
polluted	GPRegressor	0.271 ± 0.038	0.282 ± 0.045	0.282 ± 0.045
polluted	SVMRegressor	0.272 ± 0.039	0.283 ± 0.045	0.283 ± 0.046
polluted	KNNRegressor	0.333 ± 0.039	0.340 ± 0.039	0.320 ± 0.046
polluted	MLPRegressor	0.277 ± 0.040	0.288 ± 0.045	0.286 ± 0.046
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004

Table 4.13: D2: test nRMSE (lower is better) against fraction of non-distinct values (0.5, 0.75, 1 shown); mean ± SD over 3 seeds.

4.2.3. D3: fixed distinctness, increasing sample size

With a fixed 20% of non-distinct values in one random feature, the linear model gives 0.412±0.102, 0.514±0.052 and 0.232±0.015 at $n = 100$, 600 and 1000 (clean 0.188, 0.197, 0.199). There is no monotone size trend, and the variation is mainly between the values of n rather than between seeds (the SD over seeds is moderate and falls with n). In this experiment the data are regenerated for each n (`make_data(seed, x)`). Within a seed, the polluter (`pollution_first_second_third_experiments` in `library/pollution.py`, seeded with the per-repetition pollute seed) picks the same column index at every n, but `make_regression` draws the coefficients after X, so the true coefficients, and hence the weight of the polluted column, change with n. The curve therefore reflects the weight of the polluted feature at each n, not the sample size. The realised distinct fraction is 0.8 at every n.

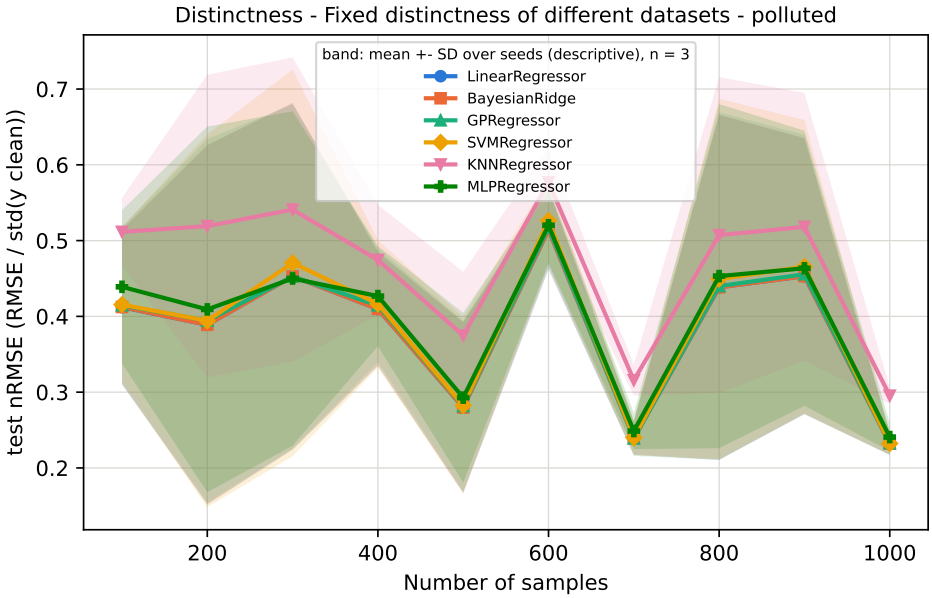


Figure 4.20: D3: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

4.2.4. D4: different percentages among features

Several of 9 features (4 informative) are polluted with percentages drawn around x. The linear model gives 0.347±0.131, 0.645±0.226 and 0.756±0.231 (clean 0.200); `KNeighborsRegressor` gives 0.551, 0.788 and 0.869. The SD over seeds is large because the set of polluted columns differs between seeds.

Condition	Model	100	600	1000
polluted	LinearRegressor	0.412 ± 0.102	0.514 ± 0.052	0.232 ± 0.015
polluted	BayesianRidge	0.412 ± 0.102	0.514 ± 0.052	0.232 ± 0.015
polluted	GPRegressor	0.415 ± 0.103	0.517 ± 0.052	0.233 ± 0.015
polluted	SVMRegressor	0.415 ± 0.102	0.526 ± 0.056	0.232 ± 0.015
polluted	KNNRegressor	0.512 ± 0.045	0.577 ± 0.045	0.295 ± 0.011
polluted	MLPRegressor	0.439 ± 0.101	0.520 ± 0.050	0.241 ± 0.018
clean	LinearRegressor	0.188 ± 0.015	0.197 ± 0.004	0.199 ± 0.004

Table 4.14: D3: test nRMSE (lower is better) against sample size n (100, 600, 1000 shown); mean ± SD over 3 seeds.

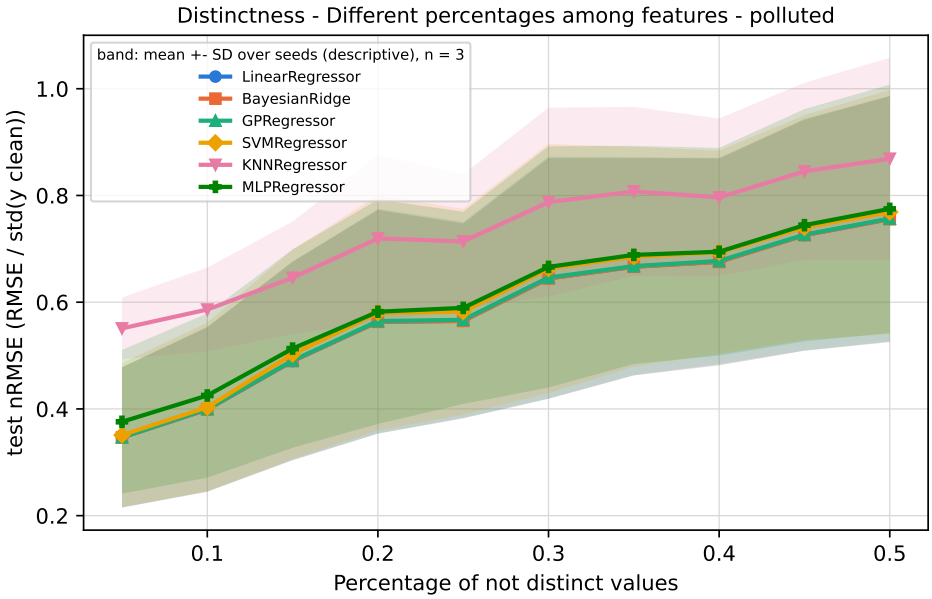


Figure 4.21: D4: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

Condition	Model	0.05	0.3	0.5
polluted	LinearRegressor	0.347 ± 0.131	0.645 ± 0.226	0.756 ± 0.231
polluted	BayesianRidge	0.347 ± 0.131	0.644 ± 0.225	0.755 ± 0.230
polluted	GPRegressor	0.347 ± 0.132	0.646 ± 0.227	0.757 ± 0.230
polluted	SVMRegressor	0.351 ± 0.135	0.663 ± 0.234	0.769 ± 0.229
polluted	KNNRegressor	0.551 ± 0.058	0.788 ± 0.177	0.869 ± 0.189
polluted	MLPRegressor	0.376 ± 0.135	0.666 ± 0.226	0.775 ± 0.233
clean	LinearRegressor	0.200 ± 0.005	0.200 ± 0.005	0.200 ± 0.005

Table 4.15: D4: test nRMSE (lower is better) against mean percentage (0.05, 0.3, 0.5 shown); mean ± SD over 3 seeds.

4.2.5. D8: high and low distinctness combined

Two random columns are polluted, with x and $1-x$ non-distinct values. At $x = 0.25$ the linear model is at 0.865 ± 0.081 and at 50% at 0.911 ± 0.034, close to the nRMSE of 1 of a mean predictor; `KNeighborsRegressor` reaches 0.948 and 0.986.

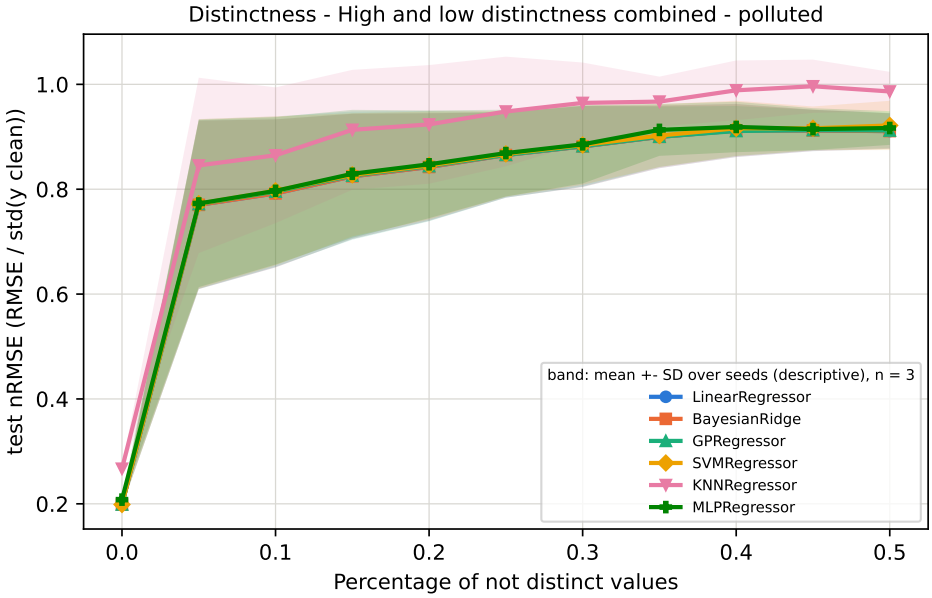


Figure 4.22: D8: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

Condition	Model	0	0.25	0.5
polluted	LinearRegressor	0.199 ± 0.004	0.865 ± 0.081	0.911 ± 0.034
polluted	BayesianRidge	0.199 ± 0.004	0.865 ± 0.081	0.911 ± 0.034
polluted	GPRegressor	0.199 ± 0.004	0.865 ± 0.082	0.912 ± 0.034
polluted	SVMRegressor	0.199 ± 0.004	0.867 ± 0.081	0.922 ± 0.047
polluted	KNNRegressor	0.267 ± 0.005	0.948 ± 0.105	0.986 ± 0.037
polluted	MLPRegressor	0.208 ± 0.005	0.869 ± 0.083	0.917 ± 0.032
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004

Table 4.16: D8: test nRMSE (lower is better) against fraction of non-distinct values x (0, 0.25, 0.5 shown); mean ± SD over 3 seeds.

4.2.6. D9: distinctness on the most informative features

The 3 most important of 9 features are polluted. The linear model goes from 0.200 to 0.698 ± 0.029 and 0.918 ± 0.025, and `KNeighborsRegressor` from 0.471 to 0.816 and

1.019 ± 0.015, i.e. worse than a mean predictor.

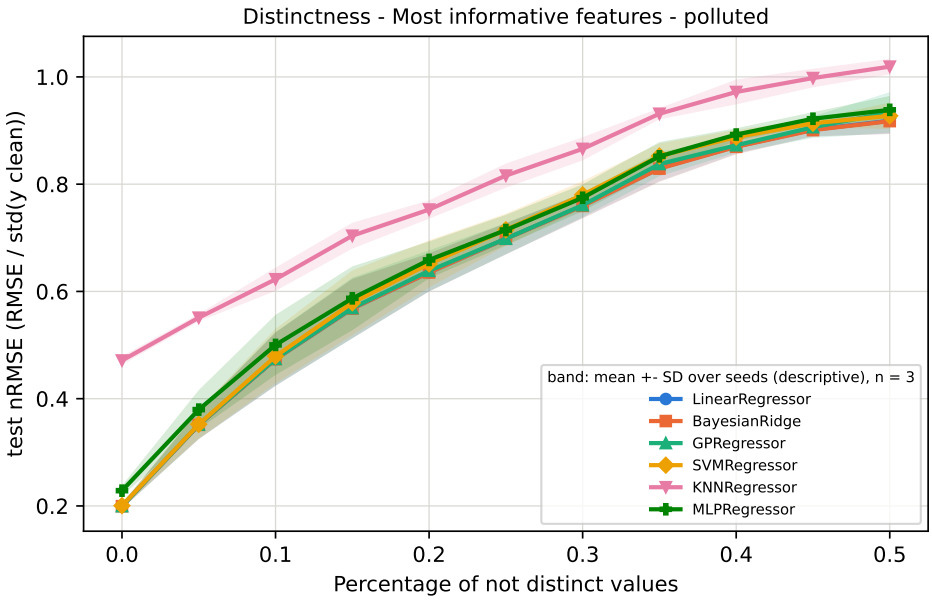


Figure 4.23: D9: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

Condition	Model	0	0.25	0.5
polluted	LinearRegressor	0.200 ± 0.005	0.698 ± 0.029	0.918 ± 0.025
polluted	BayesianRidge	0.200 ± 0.005	0.698 ± 0.029	0.918 ± 0.024
polluted	GPRegressor	0.200 ± 0.005	0.698 ± 0.029	0.934 ± 0.038
polluted	SVMRegressor	0.201 ± 0.004	0.715 ± 0.030	0.927 ± 0.025
polluted	KNNRegressor	0.471 ± 0.008	0.816 ± 0.023	1.019 ± 0.015
polluted	MLPRegressor	0.228 ± 0.009	0.714 ± 0.028	0.939 ± 0.026
clean	LinearRegressor	0.200 ± 0.005	0.200 ± 0.005	0.200 ± 0.005

Table 4.17: D9: test nRMSE (lower is better) against fraction of non-distinct values (0, 0.25, 0.5 shown); mean ± SD over 3 seeds.

4.2.7. D10: distinctness on the less informative features

The same on the 3 least important features. The linear model stays at 0.200, 0.200, 0.199, equal to the clean reference, and so do BayesianRidge, the Gaussian process and `LinearSVR` (about 0.20). `KNeighborsRegressor` is high (0.471 at rate 0) but also essentially flat (0.479, 0.482): its level comes from the many uninformative columns that enter the distance, not from the pollution. This contradicts the original statement that the pollution of unimportant features was crucial.

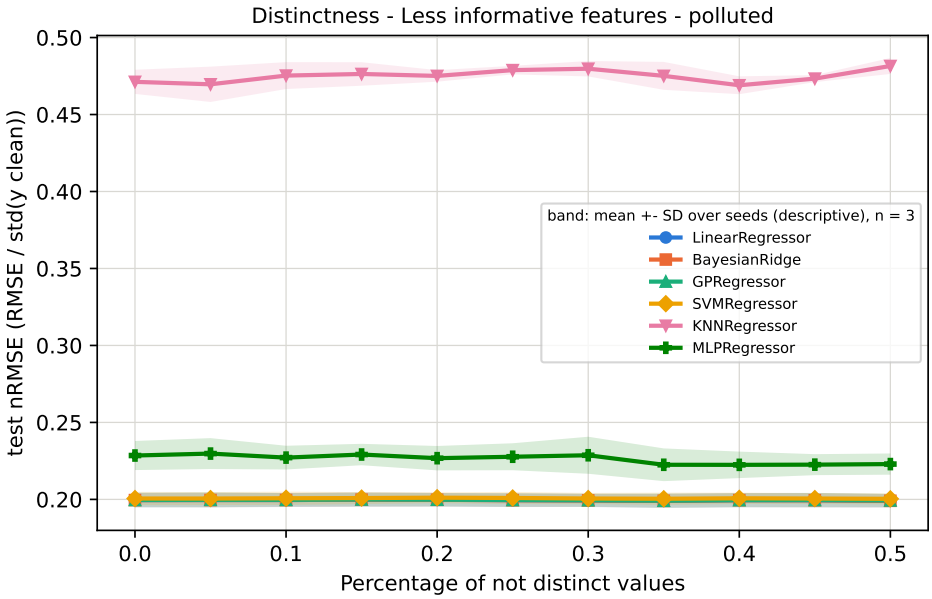


Figure 4.24: D10: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

Condition	Model	0	0.25	0.5
polluted	LinearRegressor	0.200 ± 0.005	0.200 ± 0.004	0.199 ± 0.004
polluted	BayesianRidge	0.200 ± 0.005	0.200 ± 0.004	0.199 ± 0.004
polluted	GPRegressor	0.200 ± 0.005	0.200 ± 0.004	0.199 ± 0.004
polluted	SVMRegressor	0.201 ± 0.004	0.201 ± 0.004	0.200 ± 0.004
polluted	KNNRegressor	0.471 ± 0.008	0.479 ± 0.003	0.482 ± 0.005
polluted	MLPRegressor	0.228 ± 0.009	0.228 ± 0.009	0.223 ± 0.007
clean	LinearRegressor	0.200 ± 0.005	0.200 ± 0.005	0.200 ± 0.005

Table 4.18: D10: test nRMSE (lower is better) against fraction of non-distinct values (0, 0.25, 0.5 shown); mean ± SD over 3 seeds.

Family B: duplicate records

4.2.8. D11: duplicate records

Exact copies of rows are appended (1000 original rows, 3 features). With group-aware splits (`duplicated`) the linear model is unaffected (0.199 at all rates), and `KNeighborsRegressor` rises moderately from 0.267 to 0.274 ± 0.004 and 0.283 ± 0.006 because copies re-weight neighbours. Deduplicating the training fold removes this effect (0.267 throughout). With the leaky `naive_split` the linear model is still unchanged, but `KNeighborsRegressor` appears to improve from 0.267 to 0.251 ± 0.002 and 0.223 ± 0.002 because copies of training rows are scored in the test fold: an optimistic, invalid estimate.

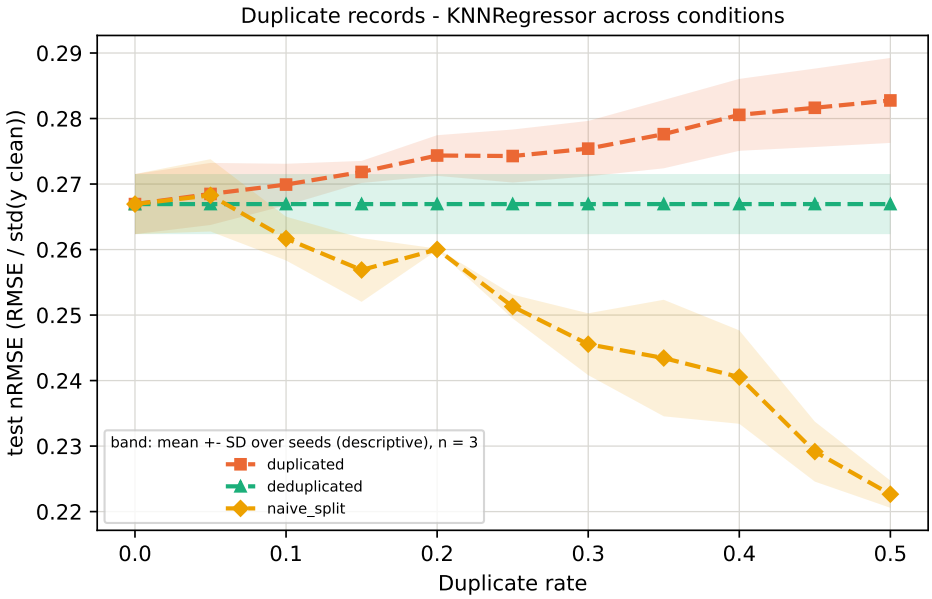


Figure 4.25: D11: test nRMSE of `KNNRegressor` for each condition, mean ± SD over 3 seeds.

Condition	Model	0	0.25	0.5
duplicated	LinearRegressor	0.199 ± 0.004	0.199 ± 0.003	0.199 ± 0.004
duplicated	KNNRegressor	0.267 ± 0.005	0.274 ± 0.004	0.283 ± 0.006
deduplicated	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004
deduplicated	KNNRegressor	0.267 ± 0.005	0.267 ± 0.005	0.267 ± 0.005
naive_split	LinearRegressor	0.199 ± 0.004	0.199 ± 0.007	0.198 ± 0.005
naive_split	KNNRegressor	0.267 ± 0.005	0.251 ± 0.002	0.223 ± 0.002

Table 4.19: D11: test nRMSE (lower is better) against duplicate rate (0, 0.25, 0.5 shown); mean ± SD over 3 seeds.

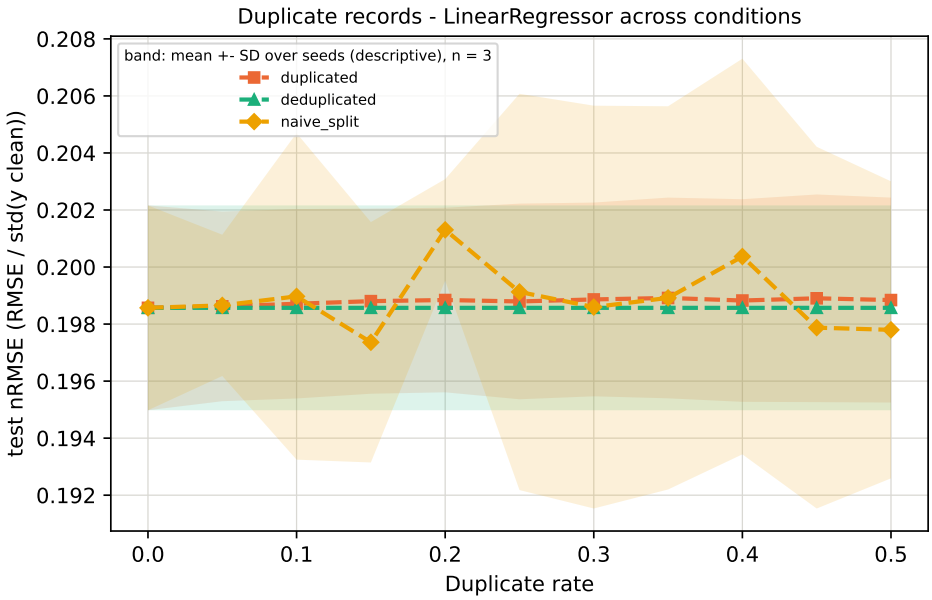


Figure 4.26: D11: test nRMSE of `LinearRegressor` for each condition, mean ± SD over 3 seeds.

Family C: accuracy and validity

4.2.9. D5: random noise on the features (accuracy)

Gaussian noise is added to all cells. The linear model degrades smoothly, 0.216 ± 0.005, 0.279 ± 0.008 and 0.318 ± 0.010 (clean 0.199), all models by almost the same amount (`KNeighborsRegressor` 0.282, 0.337, 0.377). This is the usual attenuation by errors in the features.

Condition	Model	0.00625	0.0375	0.0625
polluted	LinearRegressor	0.216 ± 0.005	0.279 ± 0.008	0.318 ± 0.010
polluted	BayesianRidge	0.216 ± 0.005	0.279 ± 0.008	0.318 ± 0.010
polluted	GPRegressor	0.216 ± 0.005	0.279 ± 0.008	0.318 ± 0.010
polluted	SVMRegressor	0.217 ± 0.004	0.279 ± 0.008	0.318 ± 0.010
polluted	KNNRegressor	0.282 ± 0.002	0.337 ± 0.004	0.377 ± 0.005
polluted	MLPRegressor	0.225 ± 0.006	0.286 ± 0.009	0.325 ± 0.011
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004

Table 4.20: D5: test nRMSE (lower is better) against noise variance (0.00625, 0.0375, 0.0625 shown); mean ± SD over 3 seeds.

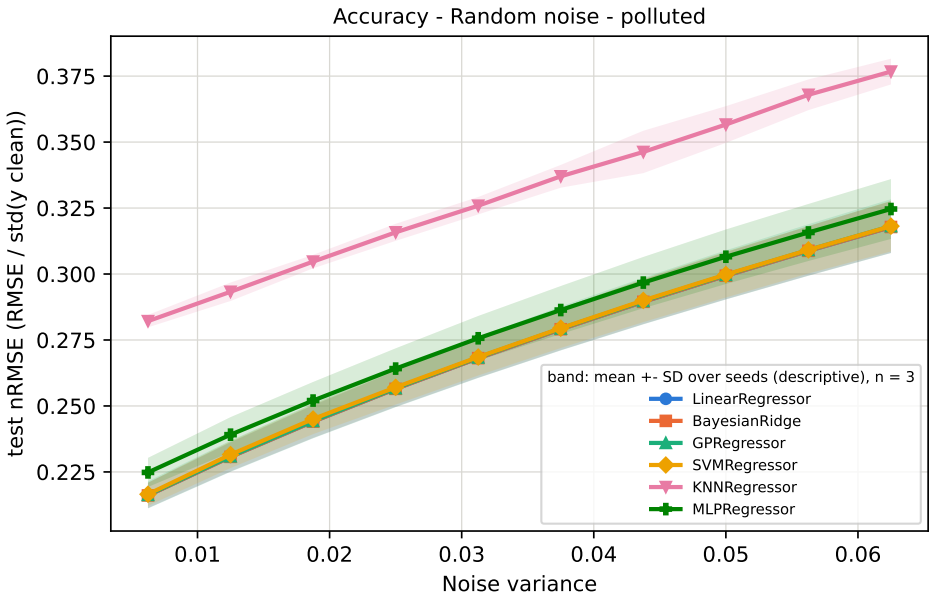


Figure 4.27: D5: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

4.2.10. D6: binning one feature (precision)

With 1 bin the feature is constant and the linear model is at 0.630 ± 0.233; with 6 bins it recovers to 0.265 ± 0.044 and with 10 bins to 0.229 ± 0.019, still above the clean 0.199. Binning always loses information; the apparent improvement with more bins is the recovery of that information, not a benefit of discretisation.

Condition	Model	1	6	10
polluted	LinearRegressor	0.630 ± 0.233	0.265 ± 0.044	0.229 ± 0.019
polluted	BayesianRidge	0.630 ± 0.233	0.265 ± 0.044	0.229 ± 0.019
polluted	GPRegressor	0.632 ± 0.233	0.265 ± 0.044	0.229 ± 0.019
polluted	SVMRegressor	0.631 ± 0.233	0.266 ± 0.044	0.230 ± 0.019
polluted	KNNRegressor	0.693 ± 0.249	0.326 ± 0.041	0.292 ± 0.012
polluted	MLPRegressor	0.633 ± 0.232	0.272 ± 0.046	0.237 ± 0.020
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004

Table 4.21: D6: test nRMSE (lower is better) against number of bins (1, 6, 10 shown); mean ± SD over 3 seeds.

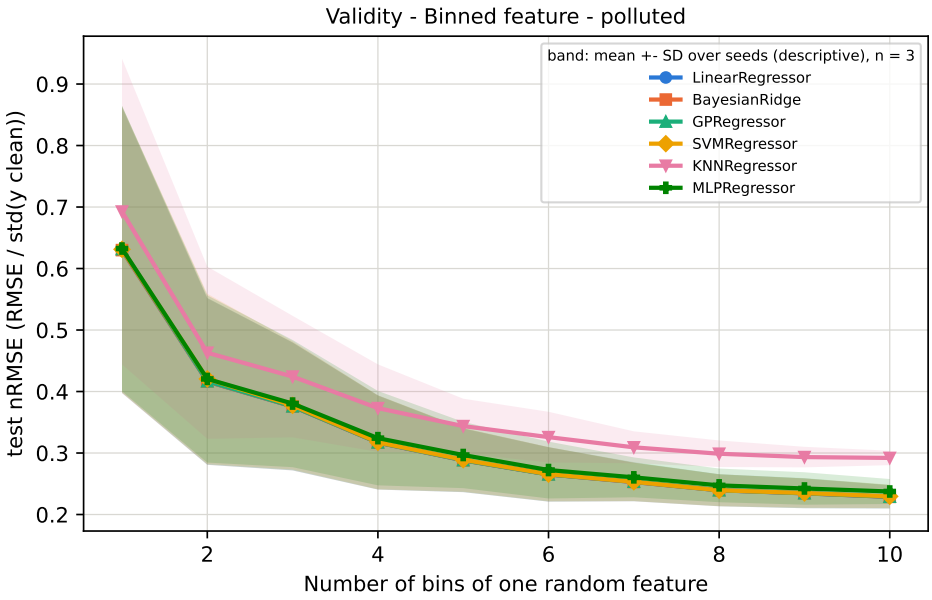


Figure 4.28: D6: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

4.2.11. D7: outliers (validity)

A random cell in the chosen rows is replaced by a random integer in $[0, 100)$, up to about 100 standard deviations of the Gaussian features, an extreme contamination. The linear model, BayesianRidge and `LinearSVR` fail: 1.256 ± 0.305, 0.992 and 0.996 for the linear model, 2.397 ± 0.189 at 1.25% for `LinearSVR`, i.e. no better than (or worse than) a mean predictor. The Gaussian process (0.242, 0.376, 0.452) and `KNeighborsRegressor` (0.315, 0.399, 0.479) degrade gracefully, and the MLP lies in between (0.527 ± 0.229, 0.337, 0.399).

Condition	Model	0.0125	0.075	0.125
polluted	LinearRegressor	1.256 ± 0.305	0.992 ± 0.006	0.996 ± 0.004
polluted	BayesianRidge	1.254 ± 0.305	0.993 ± 0.007	0.996 ± 0.005
polluted	GPRegressor	0.242 ± 0.031	0.376 ± 0.027	0.452 ± 0.031
polluted	SVMRegressor	2.397 ± 0.189	0.997 ± 0.010	0.998 ± 0.002
polluted	KNNRegressor	0.315 ± 0.047	0.399 ± 0.007	0.479 ± 0.010
polluted	MLPRegressor	0.527 ± 0.229	0.337 ± 0.015	0.399 ± 0.057
clean	LinearRegressor	0.199 ± 0.004	0.199 ± 0.004	0.199 ± 0.004

Table 4.22: D7: test nRMSE (lower is better) against fraction of rows with an outlier (0.0125, 0.075, 0.125 shown); mean ± SD over 3 seeds.

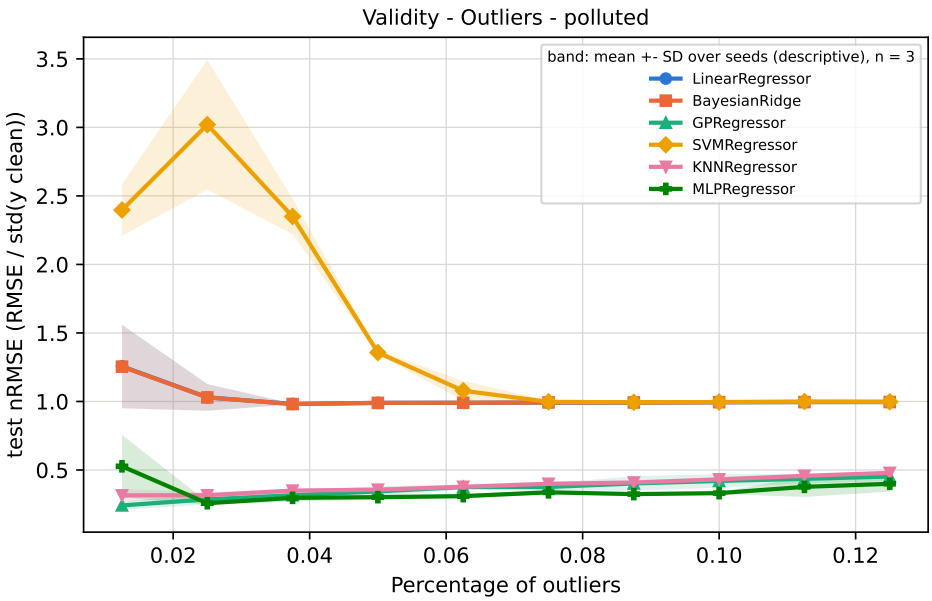


Figure 4.29: D7: test nRMSE of each model, condition `polluted`, mean ± SD over 3 seeds.

5 | Conclusions and limitations

5.1. Conclusions

- **MCAR.** Mean, median, iterative and mean-plus-indicator imputation are equivalent because the features are independent and the other columns carry no information about the missing one (Tables 4.2 and 4.3). `most_frequent` is degenerate, `std_constant` is clearly worse, and `knn` is slightly worse. The error grows steadily with the missing rate.
- **Only informative features matter.** Missing values in noise features leave the error at the clean level (C3 rank 5 and 9, C8), while the same rate in informative features is costly (C3 rank 0, C7, C9). D10 shows the same for distinctness.
- **MAR** is mild at the rates tested (Table 4.6), though only one column is affected so this is not like-for-like with MCAR on all features, with a large spread between seeds.
- **MNAR.** Mean and iterative imputation are biased; the missing-value indicator recovers a large part of the loss and the native boosting model behaves similarly (Table 4.7). In the combined MCAR plus MNAR setting the advantage of the indicator disappears at high rates (Table 4.11).
- **Duplicates** do not hurt the linear model with group-aware splits; `KNeighborsRegressor` is slightly affected and deduplication of the training fold removes the effect, while a naive split inflates the apparent performance of `KNeighborsRegressor` (Table 4.19).
- **Outliers** break the linear model, BayesianRidge and `LinearSVR`, which fall to or below the level of a mean predictor, while the Gaussian process and `KNeighborsRegressor` are more robust (Table 4.22). Noise and binning degrade all models smoothly (D5, D6).
- **Substitution.** Pasting a value into a column behaves as an accuracy error: the effect scales with feature importance and saturates (D1, D2; D8 is near the ceiling).

- **Sample size.** The sample-size experiments with pollution (C4, D3) show no consistent size trend (C4 with a large seed variance, D3 with variation between n values that reflects the weight of the polluted feature, which changes with n because the data are regenerated); only the clean learning curve (C0) shows a size effect, mainly for `KNeighborsRegressor`.

5.2. Limitations

- Only 3 seeds are used, so the SD is a rough descriptive spread and not a confidence interval.
- The data are synthetic with independent Gaussian features; the conclusion that simple imputers are equivalent depends on this and would not hold for correlated features.
- `MLPRegressor` and `LinearSVR` may report convergence warnings, which are recorded in the fold status and not suppressed.
- In C9 the groups of features share row sets with C7, and in C4 the missing-row sets of the columns are nested.
- The figures and tables are generated from the saved results (`Plot/results`) by `Report/tools/build_assets.py`; they are only as current as the last notebook run.
- The tables show three points of each x axis; the complete series are in the figures and in the results files.

Bibliography

[1] **GitHub repository that contains all the experiments mentioned in the document** https://github.com/BiancaSavoiu/DIQproject

[2] D. B. Rubin. Inference and missing data. *Biometrika*, 63(3):581–592, 1976.

[3] R. J. A. Little and D. B. Rubin. *Statistical Analysis with Missing Data*, 3rd edition. Wiley, 2019.

[4] J. Josse, N. Prost, E. Scornet and G. Varoquaux. On the consistency of supervised learning with missing values. arXiv:1902.06931, 2019.

[5] C. Batini and M. Scannapieco. *Data and Information Quality*. Springer, 2016.

[6] F. Pedregosa et al. Scikit-learn: Machine learning in Python. *Journal of Machine Learning Research*, 12:2825–2830, 2011.

[7] C. E. Rasmussen and C. K. I. Williams. *Gaussian Processes for Machine Learning*. MIT Press, 2006.

[8] S. Kaufman, S. Rosset, C. Perlich and O. Stitelman. Leakage in data mining. *ACM Transactions on Knowledge Discovery from Data*, 6(4), 2012.